

SemLink

Automatic Semantic Note Linking

BCSE332L

Deep Learning

Anumeya Sehgal (23BAI1203)

Kanishq Tiwari (23BAI1150)



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Under the guidance of
Thomas Abraham J

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore

April, 2026

Problem Statement

Graph-based note systems rely heavily on manual linking, which fundamentally does not scale with large or long-term note collections. As a result, relationships

between conceptually related notes remain unexpressed, severely limiting the usefulness of graph visualizations. Furthermore, similar automated solutions aimed at "document linking" for human interpretability consistently fall short by focusing on purely keyword-matching approaches. These rigid, lexical methods fail to capture underlying semantic meanings, resulting in incomplete networks that miss nuanced connections.

Objective

The primary objective of SemLink is to eliminate the friction of manual knowledge management by automating the discovery of meaningful connections within a note vault. Specifically, this project aims to achieve the following core goals:

- **Infer Semantic Relationships:** Automatically identify deep, contextual connections between unstructured textual notes using Natural Language Processing (NLP), completely removing the need for manual link generation.
- **Compare Similarity Methods:** Evaluate and contrast the effectiveness of traditional, keyword-based baseline models (TF-IDF) against advanced neural embedding architectures (Sentence-BERT and OpenAI API) for link discovery.
- **Construct a Sparse, Interpretable Graph:** Generate a highly readable knowledge graph by intelligently filtering weak connections (e.g., via similarity thresholds and K-Nearest Neighbor strategies), ensuring the resulting visualization prioritizes structural discovery over visual clutter.
- **Analyze Graph Structure and Semantic Coherence:** Compute complex graph metrics and apply community detection algorithms to evaluate the network's structural integrity and automatically aggregate related notes into distinct topic clusters.
- **Provide an Accessible CLI Tool:** Deliver a robust Command Line Interface (CLI) that packages the entire data pipeline—from ingestion to visualization—into an intuitive tool, allowing for easy usage and incremental vault synchronization.

Selected Deep Learning Approach

To capture semantic similarity accurately and overcome the brittle nature of exact-keyword matching, SemLink extends beyond purely lexical approaches by introducing a robust, dual-backend neural embedding architecture. This design provides the flexibility to prioritize either local data privacy or maximum semantic resolution.

- **Baseline vs. Neural Architectures:** While the system retains a scikit-learn TF-IDF baseline as its default for lightweight, CPU-only execution, its core deep learning capabilities rely on transformer-based dense vector representations.
- **Local Processing (Sentence-BERT):** For rapid, offline, and privacy-preserving inference, the system integrates the sentence-transformers library backed by PyTorch. By default, it deploys the all-MiniLM-L6-v2 model, mapping texts into a 384-dimensional vector space. For higher quality semantic mapping at the cost of slightly increased processing time, the architecture also supports the heavier, 768-dimensional all-mpnet-base-v2 model.
- **Cloud-Accelerated High-Fidelity:** For state-of-the-art semantic resolution, SemLink interfaces directly with OpenAI's cloud-based embedding endpoints. Utilizing the text-embedding-3-small or text-embedding-3-large models, the system generates ultra-dense, 1536-dimensional or 3072-dimensional embeddings, allowing for the detection of highly abstract conceptual overlaps across disparate academic subjects.
- **LLM Topic Label Generation:** To generate descriptive topic labels for detected clusters, the system uses OpenAI GPT-4o-mini. This model analyzes note titles and content excerpts from each cluster and produces a 2-4 word descriptive label (e.g., "Computer Networks", "Machine Learning").
- **Mathematical Similarity Computation:** Regardless of the backend model selected, the high-dimensional vectors are evaluated for conceptual overlap using Cosine Similarity. The embeddings undergo L2 normalization, and the system computes a pairwise similarity matrix (S , where n is the total number of notes) to assign a quantitative confidence score to every potential link in the vault.

Dataset Description

The dataset utilized for evaluating this methodology is a custom, locally hosted repository consisting of the personal academic notes and reference materials accumulated by the project's primary collaborator over the duration of their undergraduate degree.

- **Volume and Evolution:** Unlike static, pre-packaged academic datasets (like ArXiv abstracts), this dataset represents an organic, evolving "second brain" developed through personal knowledge management. Because it is a genuine personal vault, it contains a high degree of variance in note length, structure, and formatting.
- **Multimodal Document Formats:** Because academic study is rarely confined to a single format, the dataset is inherently heterogeneous. It primarily consists of unstructured Markdown (.md) files representing the developer's daily lecture notes, conceptual summaries, and coding snippets.
- **External Artifacts:** To capture a realistic, comprehensive research environment, the dataset also incorporates raw text (.txt), portable document formats (.pdf) representing published research papers and textbooks, and Microsoft Word documents (.docx) representing drafted reports and university assignments. This multimodality requires an aggressive, unified ingestion pipeline to standardize the text before it reaches the deep learning models.

Methodology

The core methodology involves a sequential, decoupled data pipeline that transitions unstructured local files into an interconnected knowledge graph. The system ingests notes from a specified vault directory, generates dense semantic embeddings, infers topological links based on mathematical similarity, analyzes the resulting graph metrics, and finally exports the network for visualization. Because neural embedding models are highly sensitive to formatting noise and context length, the foundation of this entire pipeline relies on an aggressive, highly robust data ingestion and text normalization architecture.

Data Preprocessing Steps

File Discovery and Noise Exclusion: The ingestion engine utilizes a top-down directory traversal algorithm (`os.walk`) to map the vault. To prevent the system from processing system artifacts or software dependencies, it explicitly bypasses high-priority exclusion directories (such as `.git`, `node_modules`, **`pycache`**, and all hidden `.folders`). Furthermore, it supports a `.semLinkIgnore` file for custom gitignore-style patterns. It employs pattern matching (`fnmatch`) to filter out semantic noise by ignoring non-content files like `LICENSE*`, package locks (`.lock`), and compiled binaries (`.pyc`, `*.exe`, `*.dll`).

Format-Specific Content Extraction: Because the vault contains multimodal document types, the system applies specific extraction logic based on file extensions:

- **PDFs:** The system utilizes the pypdf library to iterate through the document, extracting raw text from all pages. These pages are then concatenated using double newlines to programmatically simulate standard paragraph structures for the embedding model.
- **Word Documents (.docx):** Using the python-docx library, the system iterates through the document's XML structure, extracting only the paragraphs that actively contain text strings, strategically skipping empty formatting blocks, images, and layout elements.
- **Markdown and Plain Text (.md, .txt):** To prevent runtime crashes caused by mixed-origin files, these documents are loaded using a robust, multi-pass encoding detection sequence. The system attempts to read the file as utf-8, followed by utf-8-sig, latin-1, and cp1252. If all standard encodings fail, it implements a fallback mechanism that reads the file while replacing corrupted characters to ensure the pipeline is not halted by a single malformed file.

Semantic Markdown Stripping: Raw markdown text cannot be fed directly into an embedding model without introducing syntax noise (e.g., asterisks, hash symbols, brackets). Instead of using brittle Regular Expressions for this task, SemLink parses the text into an Abstract Syntax Tree (AST) using the markdown-it-py CommonMark parser. A custom traversal function then recursively walks through these tokens. It strips away structural tags (like paragraph_open or list_item_open) but explicitly extracts and preserves the internal text, inline code snippets (code_inline), and the contents of fenced code blocks (fence).

Aggressive Text Normalization: Once the markup is stripped, the plain text undergoes strict normalization to ensure consistent vector representations:

- The text is subjected to Unicode normalization (NFKC) to decompose and recompose characters into a standardized format.
- All characters are converted to lowercase.
- Erratic whitespace is sanitized: tabs and consecutive spaces are collapsed into a single space, trailing whitespaces are stripped from individual lines, and excessive vertical spacing (three or more consecutive newlines) is collapsed into a standard double-newline paragraph break.

Targeted Metadata Extraction: Concurrent with the content cleaning process, the system utilizes compiled Regular Expressions to extract highly valuable structural metadata directly from the raw note. This includes extracting all internal headings ($\wedge\#\{1,6\}$), isolating wiki-style links ([link](#)), and calculating the exact word count of the cleaned text. If a note lacks a primary heading, the system falls back to using the file's stem name as the document title.

Data Structuring and Persistence: Finally, the fully normalized content and its corresponding metadata are packaged into a Python Note dataclass. To support incremental vault updates (ensuring files are not re-processed unnecessarily),

each note is assigned a highly stable, deterministic MD5 hash ID derived from its absolute file path. These objects are then cached in memory within a NoteStore and serialized to disk using orjson for high-performance JSON persistence.

Model Architecture

Embedding Models

Model	Dimensions	Type	Use Case
TF-IDF	Variable	Traditional	Baseline, CPU-only
all-MiniLM-L6-v2	384	Sentence-BERT	Local neural
all-mpnet-base-v2	768	Sentence-BERT	Higher quality
text-embedding-3-small	1536	OpenAI	Cloud high-fidelity
text-embedding-3-large	3072	OpenAI	Cloud maximum quality

Topic Generation

Model	Type	Use Case
gpt-4o-mini	OpenAI LLM	Topic label generation
Centeroid	Page ranking and Centeroid identification	Topic label generation for small knowledge bases with good naming conventions

Link Inference Strategies

Strategy	Description	Graph Density
Threshold	Connect if similarity $\geq$ threshold	Sparse
KNN	Connect to k nearest neighbors	Moderate
Mutual KNN	Connect only if mutually nearest	Sparse

Strategy	Description	Graph Density
Hybrid	KNN + threshold (recommended)	Balanced

5.3 Tools and Technologies Used

Component	Technology
CLI Framework	Typer
Display	Rich
Preprocessing	markdown-it-py, pypdf, python-docx
ML/Scikit-learn	TF-IDF, sklearn
Neural	sentence-transformers, openai
Graph	NetworkX
Community Detection	python-louvain
Storage	SQLite (orjson)
API	FastAPI
Visualization	D3.js (React frontend)

Training Procedure

This project does not involve model training in the traditional sense. Instead, it utilizes:

1. **Pre-trained Models:** Uses pre-trained transformer models (SBERT, OpenAI embeddings)
2. **Fine-tuned via Similarity:** Link inference based on cosine similarity
3. **LLM for Labels:** Uses GPT-4o-mini for topic label generation

The system processes notes in batches:

- OpenAI embeddings: 100 notes per API call
- Similarity matrix: Pairwise computation using sklearn

- Community detection: Louvain algorithm optimization

Hyperparameter Settings

Parameter	Default	Description
--method	tfidf	Embedding method (tfidf/sbert/openai)
--min-weight	0.1	Minimum similarity threshold
--k	7	K-nearest neighbors
--resolution	1.0	Louvain community resolution
--topic-method	llm	Topic generation (llm/keywords)
--topic-model	gpt-4o-mini	LLM model for topics

Evaluation Metrics

Metric	Description
Precision@K	Fraction of true positives in top-K predictions
Recall@K	Coverage of actual relationships in top-K
Modularity	Graph community quality score
Coverage	Percentage of notes with connections
Topic Coherence	Manual evaluation of topic clusters
Edge Weight Distribution	Distribution of similarity scores

Results and Output Screenshots / Graphs

CLI Output

```
$ semlink run ./notes --output ./output
[INFO] Discovered 150 notes
```

[INFO] Generated embeddings (method: openai)
[INFO] Inferred 423 links (strategy: hybrid)
[INFO] Topics: 12 clusters detected
[INFO] Exported graph.json

Topic Output Example

Topic ID	Label	Notes	Keywords
0	Computer Networks	15	network, protocol, ip, tcp
1	Machine Learning	12	model, training, neural, gradient
2	Data Structures	8	array, tree, algorithm, complexity

Web Interface

The React + D3.js frontend provides:

- Interactive force-directed graph visualization
- Node click to show connections and content
- Topic panel for filtering by detected clusters
- Control panel for filtering by weight and connections

Performance Analysis / Discussion

Embedding Method Comparison

Method	Speed	Quality	Resources
TF-IDF	Fastest	Baseline	Low (CPU)
SBERT	Moderate	Good	Medium (GPU)
OpenAI	Fast	Best	API cost

Link Strategy Comparison

Strategy	Graph Density	Interpretability
Threshold	Sparse	High (clear cutoffs)

Strategy	Graph Density	Interpretability
KNN	Moderate	Medium
Hybrid	Balanced	High

Topic Generation Quality

The LLM-based topic generation significantly outperforms keyword-based approaches:

- **Keyword-based:** Produces generic labels like "Model & State & Learning"
- **LLM-based:** Produces descriptive labels like "Computer Networks", "Machine Learning"

Challenges Addressed

1. **Poor Ignoring:** Added .semLinkIgnore support for custom patterns
2. **Poor Topic Headings:** Now using LLM to generate descriptive labels
3. **Binary Content:** Added text filtering before API calls to prevent errors

Conclusion

SemLink successfully demonstrates automatic semantic note linking using deep learning approaches. The key achievements include:

1. **Automated Discovery:** No manual linking required—relationships are inferred from content
2. **Multiple Methods:** Comparison of TF-IDF baseline vs neural embeddings vs OpenAI
3. **Interpretable Graph:** Filtered connections via threshold and KNN strategies
4. **Topic Clusters:** LLM-generated descriptive topic labels using GPT-4o-mini
5. **Accessible CLI:** Complete pipeline from ingestion to visualization

The comparison between embedding methods shows clear quality improvements:

- TF-IDF provides fast baseline results with minimal resources
- SBERT captures semantic similarity better than TF-IDF
- OpenAI provides highest quality with cloud resources
- LLM topic generation produces human-readable descriptive labels

The system successfully addresses manual note-linking fatigue by automatically discovering meaningful relationships and presenting them as an interactive knowledge graph.

Appendix I: Outputs

Stage	Output	Purpose
Ingestion	notes.json	Acts as the initial input for all text processing. Structured JSON file
Embedding generation	embeddings.npz	Embeddings over the complete input corpus
Linking	graph.json	Structured graph object in format as required for frontend generation
Aggregation	Modified graph.json	This step alters the graph.json file to add color for identified topics and trends
Topic Generation	Aggregated topic list (endpoint)	Returns beautified titles for inferred topics



Figure 1: Semlink Output - Engineering Resources

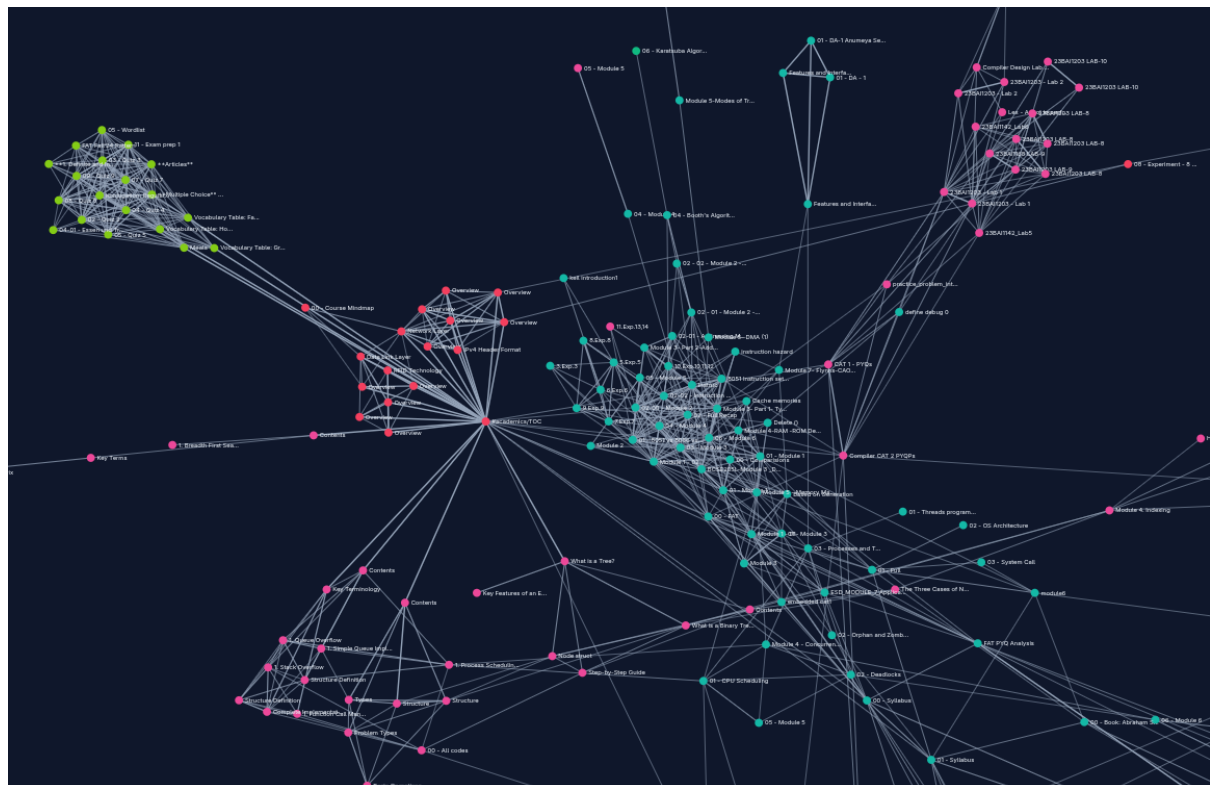


Figure 2: Graph view

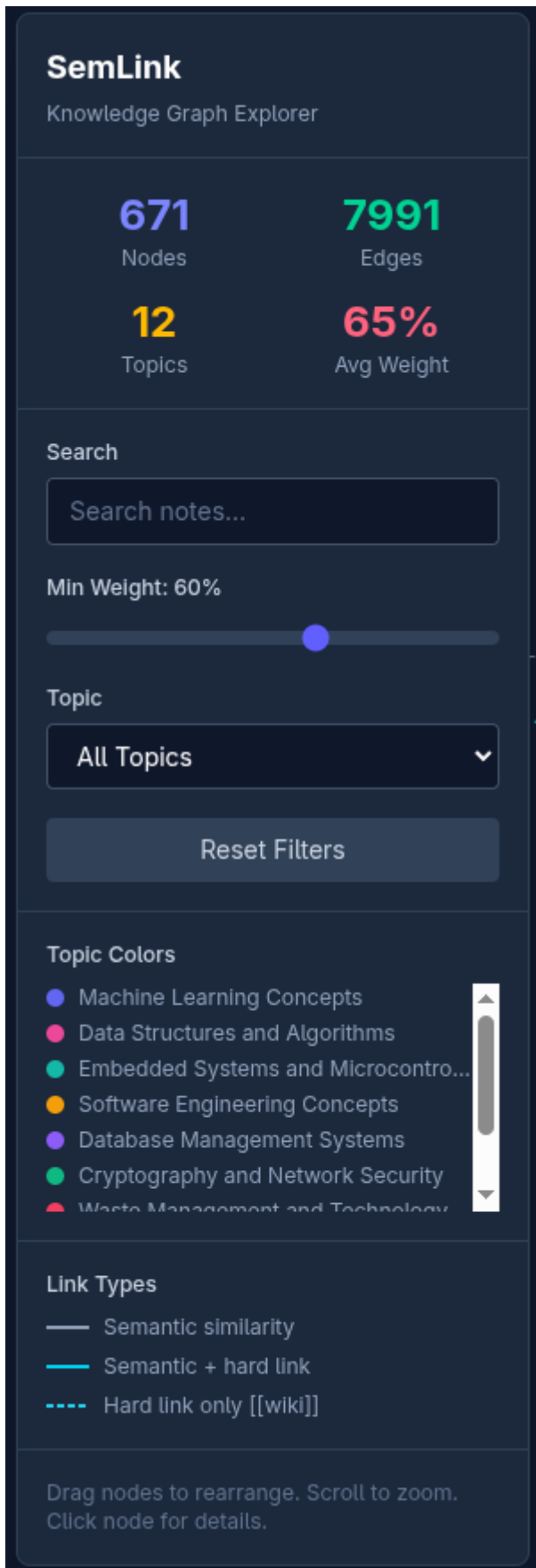


Figure 3: Documents parsed and topics generated

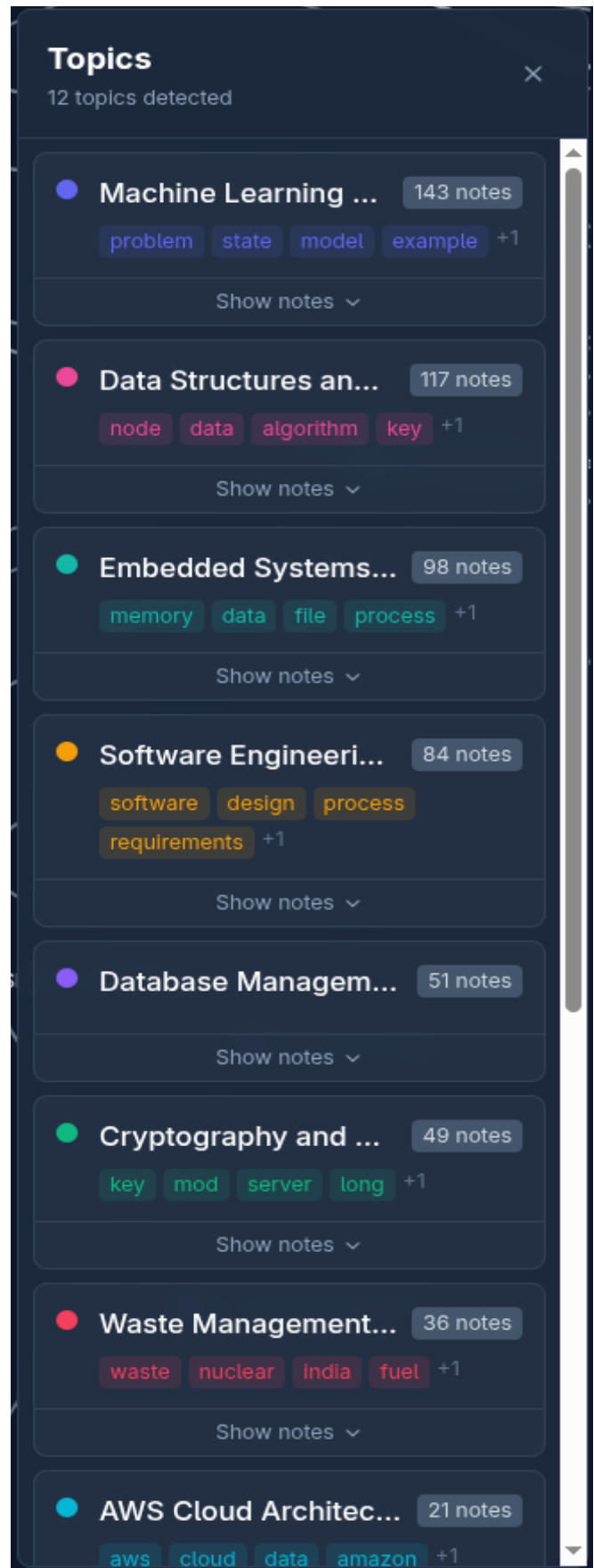


Figure 4: List of all topics

Appendix II: Source Code

The complete source code is available on the project github:

<https://github.com/kreativethinker/semlink.git>

Project Structure

```
SemLink/
├── src/semlink/
│   ├── core/
│   │   ├── ingest.py          # Note discovery & preprocessing
│   │   ├── tfidf.py           # TF-IDF embeddings
│   │   ├── embeddings.py      # SBERT & OpenAI embeddings
│   │   ├── linker.py          # Link inference strategies
│   │   ├── graph.py           # NetworkX graph building
│   │   ├── analysis.py        # Metrics, community detection
│   │   ├── aggregate.py       # Topic aggregation
│   │   ├── topic_llm.py       # LLM topic generation
│   │   ├── visualize.py       # Export options
│   │   ├── storage.py         # SQLite persistence
│   │   └── evaluate.py        # Method comparison
│   ├── server.py              # FastAPI backend
│   └── cli.py                 # Typer CLI
├── frontend/                  # React + D3.js web interface
├── pyproject.toml
├── README.md
└── report.md
```

Key Dependencies

```
dependencies = [
    "rich>=14.0.0",
    "typer>=0.15.0",
    "markdown-it-py>=3.0.0",
    "pypdf>=4.2.0",
    "python-docx>=1.1.0",
    "scikit-learn>=1.5.0",
    "numpy>=2.0.0",
    "networkx>=3.3",
    "orjson>=3.10.0",
    "uvicorn>=0.41.0",
    "fastapi>=0.135.1",
    "openai>=2.24.0",
]
```

Installation

```
# Full installation
pip install semlink[all]

# Or from source
git clone https://github.com/KreativeThinker/SemLink.git
cd SemLink
uv sync
```

Usage Examples

```
# Full pipeline with OpenAI embeddings and LLM topic generation
semlink run ./vault --method openai --topic-method llm

# Topic aggregation with LLM labels
semlink aggregate graph.json --notes notes.json --topic-method llm

# Web interface
semlink serve --db .semlink.db
```